

INCIDENT RESPONSE REPORT

1. Introduction

Purpose of Simulation

The purpose of this security simulation was to emulate real-world cyberattack scenarios in a controlled lab environment. The exercise aimed to:

- Identify vulnerabilities in a deliberately insecure web application
- Simulate attacker behavior using common offensive tools
- Monitor and detect malicious activities using a SIEM solution
- Apply defensive controls to mitigate attacks
- Evaluate the effectiveness of detection and response mechanisms

Tools Used

- **DVWA (Damn Vulnerable Web Application):** Used as the target web application containing known vulnerabilities
- **Kali Linux:** Used as the attacker machine for reconnaissance and exploitation
- **Wazuh:** Used as a Security Information and Event Management (SIEM) tool for monitoring and alerting
- **Cowrie Honeypot:** Used to simulate a vulnerable SSH service and capture attacker behavior
- **ModSecurity (WAF):** Used to protect the web server from malicious requests
- **UFW Firewall (Uncomplicated):** Used for network-level traffic filtering

2. Network Architecture

Lab Setup

The environment consisted of three main systems:

- **Kali Linux (Attacker):** 192.168.56.4
- **Ubuntu Server (Target):** 192.168.56.8
 - Hosts DVWA (web application)
 - Hosts Cowrie (honeypot on port 2222)
- **Wazuh Server (Monitoring):** 192.168.56.7

Diagram (Text Representation)

```
Kali Linux (Attacker)
    |
    | (Nmap, Metasploit, and Hydra)
    v
Ubuntu Server (DVWA + Cowrie Honeypot)
    |
```

| (Logs & Events)
v
Wazuh SIEM (Monitoring & Alerts)

3. Attack Phase

Reconnaissance (Nmap Findings)

- i) Command used: `nmap -sS -sV -A 192.168.56.8`
- ii) Key findings:
 - Open HTTP port (80) hosting DVWA
 - Open SSH-like service on port 2222 (Cowrie honeypot)
 - Service and version detection revealed potential attack vectors

Exploits Used

i) Metasploit Web Delivery Exploit

- Payload: `python/meterpreter/reverse_http`
- Gained reverse shell access to target

ii) Manual Web Exploitation

- SQL Injection: `1' OR '1'='1`
- Cross-Site Scripting (XSS): `<script>alert('Hacked')</script>`
- Command Injection: Executed system-level commands via DVWA input field

Vulnerabilities Identified

- Input validation flaws (SQL Injection, Command Injection)
- Lack of output encoding (XSS)
- Weak authentication controls
- Exposed services (HTTP, SSH honeypot)
- Misconfigured application security settings (DVWA set to low security)

4. Detection & Monitoring

Wazuh Alerts

Wazuh successfully detected:

- Port scanning activity from Nmap
- Brute-force login attempts via Hydra
- Suspicious web requests indicating command injection
- Honeypot interaction logs from Cowrie

Log Analysis

- Logs showed repeated connection attempts to port 2222 (honeypot)

- Command execution logs (e.g., `whoami`, `cat /etc/passwd`)
- Web attack payloads captured in HTTP logs
- Alerts correlated attacker IP (Kali machine) with malicious activity patterns

5. Response Actions

WAF Rules Applied (ModSecurity)

- Enabled **SecRuleEngine On**
- Filtered malicious HTTP requests
- Blocked common attack patterns such as:
 - SQL injection strings
 - Script tags (XSS attempts)
 - Command injection payloads

Firewall Controls

- Enabled UFW firewall
 - Allowed HTTP traffic (port 80)
 - Blocked unauthorized access:
 - Denied port 2222 (honeypot exposure control)
 - Reduced attack surface by restricting unnecessary ports
-

6. Lessons Learned

Weaknesses Discovered

- DVWA intentionally exposed critical vulnerabilities
- Lack of input validation allowed multiple exploit types
- Open ports increased exposure to scanning and brute-force attacks
- Default/weak credentials could be easily exploited
- Initial lack of WAF (Web Application Firewall) and firewall protections

Improvements Needed

- Enforce strong input validation and sanitization
- Deploy WAF in production environments
- Restrict unnecessary open ports
- Implement strong authentication policies
- Enable continuous monitoring with SIEM tools
- Regular patching and system hardening

7. Conclusion

The simulation successfully demonstrated the full lifecycle of a cyberattack, from reconnaissance to exploitation, detection, and response. It highlighted how easily vulnerabilities can be exploited in poorly secured systems and emphasized the importance of layered security controls.

The integration of monitoring tools such as Wazuh, combined with defensive mechanisms like ModSecurity and firewalls, significantly improved the detection and mitigation of threats. This exercise reinforces the need for proactive security measures, continuous monitoring, and incident response readiness in real-world environments.